

ДИСЦИПЛИНА	Фронтенд и бэкенд разработка
ИНСТИТУТ	ИПТИП
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Методические указания к практическим занятиям по дисциплине
ПРЕПОДАВАТЕЛЬ	Загородних Николай Анатольевич Краснослободцева Дарья Борисовна
СЕМЕСТР	4 семестр, 2025/2026 уч. год

Практическое занятие 23

Контейнеризация приложений с Docker

Рассмотрим понятие контейнеризации, ключевые концепции Docker, создание образов с помощью `Dockerfile`, управление многоконтейнерными приложениями через `Docker Compose`, а также развертывание реального микросервисного приложения на Node.js. Все команды выполняются в среде **WSL (Windows Subsystem for Linux)**. Решение практического задания осуществляется внутри соответствующей рабочей тетради, расположенной в СДО.

Что такое контейнеризация

Когда разработчик пишет приложение на своём компьютере, оно может прекрасно работать в его окружении. Но при попытке запустить то же приложение на другой машине — на сервере, у коллеги или в облаке — нередко возникают ошибки: несовпадение версий Node.js, Python, системных библиотек и т.д. Именно для решения этой проблемы появилась контейнеризация.

Контейнеризация — это технология упаковки приложения вместе со всеми его зависимостями (библиотеками, конфигурациями, средой выполнения) в единый изолированный блок — **контейнер**. Контейнер запускается одинаково на любой машине, где установлена поддерживающая платформа.

Принципиальное отличие контейнера от виртуальной машины:

- **Виртуальная машина (VM)** эмулирует полноценный компьютер вместе с операционной системой, что требует много ресурсов и времени на запуск.

- **Контейнер** использует ядро хостовой ОС и запускается за секунды, потребляя значительно меньше памяти и ресурсов процессора.

Что такое Docker

Docker — наиболее распространённая платформа для создания, распространения и запуска контейнеров. Docker предоставляет удобные инструменты для работы с контейнерами и целую экосистему для командной разработки.

Ключевые понятия Docker:

Образ — это неизменяемый шаблон, из которого создаётся контейнер. Образ содержит операционную систему, среду выполнения (например, Node.js), код приложения и все зависимости. Образы хранятся в реестрах — например, в [Docker Hub](#).

Контейнер — это запущенный экземпляр образа. Можно запустить несколько контейнеров из одного образа. Контейнеры изолированы друг от друга и от хостовой системы.

Dockerfile — текстовый файл с инструкциями для сборки образа. В нём описывается последовательность шагов: взять базовый образ, скопировать файлы, установить зависимости, задать команду запуска.

Docker Compose — инструмент для описания и запуска многоконтейнерных приложений. Вся конфигурация хранится в одном файле `docker-compose.yml`, после чего весь стек сервисов запускается одной командой.

Docker Hub — публичный реестр образов. На нём хранятся официальные образы Node.js, Nginx, PostgreSQL, Redis и тысячи других. Образы можно скачивать (`pull`) и загружать (`push`) в реестр.

Ключевые концепции: сети, тома, переменные окружения

Сети Docker (Networks)

По умолчанию контейнеры изолированы и не могут общаться между собой. Для организации взаимодействия между контейнерами используются **Docker-сети**. Если несколько контейнеров находятся в одной сети, они могут обращаться друг к другу по **имени сервиса** как по имени хоста.

Пример: контейнер `api_gateway` может обращаться к контейнеру `service_users` просто по адресу `http://service_users:8000` — Docker сам разрешит имя в нужный IP-адрес.

Томы Docker (Volumes)

Данные внутри контейнера **не сохраняются** после его остановки — контейнер эфемерен. Для сохранения данных между перезапусками используются **тома** — специальные области хранения, которые монтируются внутрь контейнера. Типичные случаи использования: хранение данных базы данных, логов, загружаемых файлов.

Переменные окружения (Environment Variables)

Переменные окружения позволяют передавать конфигурацию в контейнер без изменения кода. Например, строку подключения к базе данных, порт, секретный ключ. В `docker-compose.yml` они задаются через секцию `environment`, а в коде читаются через `process.env.VARIABLE_NAME` (Node.js).

Установка WSL и Docker (Windows)

Все команды Docker в данном руководстве выполняются внутри **WSL** — подсистемы Linux для Windows. Это позволяет использовать полноценный Linux-терминал без виртуальной машины.

Включение WSL в Windows

Откройте **PowerShell от имени администратора** (Win + X → «Windows PowerShell (администратор)») и выполните:

```
wsl --install
```

Эта команда автоматически установит WSL и загрузит дистрибутив Ubuntu по умолчанию. После завершения **перезагрузите компьютер**.

Если команда выше не сработала, выполните вручную:

```
dism.exe /online /enable-feature /featurename:Microsoft-Windows-Subsystem-Linux /all /norestart  
dism.exe /online /enable-feature /featurename:VirtualMachinePlatform /all /norestart
```

После выполнения — снова перезагрузите компьютер. `wsl --insta`

Первичная настройка Ubuntu

После перезагрузки автоматически откроется окно Ubuntu. Придумайте имя пользователя и пароль для WSL-окружения. При вводе пароля символы не отображаются — это нормально.

Обновите пакеты:

```
sudo apt update && sudo apt upgrade -y
```

Установка Docker внутри WSL

Выполните следующие команды последовательно в терминале WSL:

```
# Обновляем индекс пакетов
sudo apt update

# Устанавливаем необходимые пакеты для работы с репозиториями по HTTPS
sudo apt install -y ca-certificates curl

# Создаем директорию для ключей и добавляем официальный GPG-ключ Docker
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Добавляем репозиторий Docker в источники APT
echo \
  "deb [arch=$(dpkg --print-architecture) signed-
  by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Снова обновляем индекс пакетов
sudo apt update

# Устанавливаем Docker Engine, Containerd и Docker Compose Plugin
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin
```

Установка Docker Desktop (опционально, но рекомендуется)

Docker Desktop предоставляет графический интерфейс и глубокую интеграцию с WSL.

1. Перейдите на <https://www.docker.com/products/docker-desktop/> и скачайте установщик для Windows.
2. Запустите Docker Desktop Installer.exe, следуйте инструкциям. **Обязательно** установите галочку «Install required Windows components for WSL 2».
3. После установки и перезагрузки откройте Docker Desktop:
 - Перейдите в **Settings** → **General** и убедитесь, что включена опция **Use the WSL 2 based engine**.

- Перейдите в **Settings** → **Resources** → **WSL Integration** и включите интеграцию с вашим дистрибутивом Ubuntu.

Проверка установки Docker

Откройте терминал WSL (найдите «Ubuntu» в меню «Пуск») и выполните:

```
# Проверяем версию Docker
docker --version

# Проверяем, что демон работает (список запущенных контейнеров пуст)
docker ps

# Проверяем версию Docker Compose
docker compose version

# Запускаем тестовый контейнер
docker run hello-world
```

Если контейнер `hello-world` выполнился и вывел приветственное сообщение — Docker установлен корректно.

Структура Dockerfile

`Dockerfile` — это набор инструкций, по которым Docker собирает образ. Каждая инструкция создаёт отдельный **слой** образа. Docker кэширует слои, поэтому при повторной сборке пересобираются только те слои, в которых что-то изменилось.

Основные инструкции Dockerfile

Инструкция	Назначение
<code>FROM <образ></code>	Задаёт базовый образ. Любой <code>Dockerfile</code> начинается с этой инструкции. Например, <code>FROM node:18-alpine</code> берёт Node.js 18 на основе лёгкого дистрибутива Alpine Linux (~5 МБ)
<code>WORKDIR <путь></code>	Устанавливает рабочую директорию внутри контейнера. Все последующие команды выполняются в ней. Если директория не существует — создаётся автоматически
<code>COPY <src> <dst></code>	Копирует файлы с хостовой машины в контейнер. <code>COPY . .</code> копирует всё из текущей директории в <code>WORKDIR</code>
<code>ADD <src> <dst></code>	Аналог <code>COPY</code> , но дополнительно умеет распаковывать архивы <code>.tar</code> и скачивать файлы по URL. Рекомендуется использовать <code>COPY</code> там, где не нужны эти функции

Инструкция	Назначение
<code>RUN <команда></code>	Выполняет команду при сборке образа и сохраняет результат как новый слой. Используется для установки зависимостей, компиляции и т.п.
<code>ENV <ключ>=<значение></code>	Устанавливает переменную окружения, которая будет доступна как во время сборки, так и в запущенном контейнере
<code>ARG <имя></code>	Объявляет переменную, которая передаётся только во время сборки (<code>docker build --build-arg</code>). В отличие от <code>ENV</code> , недоступна в запущенном контейнере
<code>EXPOSE <порт></code>	Документирует, на каком порту контейнер принимает соединения. Сам по себе порт не открывает — это делается через <code>ports</code> в <code>docker-compose.yml</code> или флагом <code>-p</code> при <code>docker run</code>
<code>VOLUME <путь></code>	Объявляет точку монтирования тома. Данные по этому пути будут сохраняться вне контейнера
<code>CMD <команда></code>	Задаёт команду по умолчанию при запуске контейнера. Может быть переопределена при <code>docker run</code> . В <code>Dockerfile</code> должна быть только одна инструкция <code>CMD</code>
<code>ENTRYPOINT <команда></code>	Задаёт точку входа — команду, которую нельзя переопределить при <code>docker run</code> (только дополнить аргументами). Часто используется вместе с <code>CMD</code>
<code>USER <пользователь></code>	Задаёт пользователя, от имени которого будут выполняться последующие команды и запускаться контейнер. Используется для повышения безопасности
<code>LABEL <ключ>=<значение></code>	Добавляет метаданные к образу: автор, версия, описание
<code>.dockerignore</code>	Не инструкция, но важный файл рядом с <code>Dockerfile</code> : перечисляет файлы и папки, которые не будут скопированы в контейнер (аналог <code>.gitignore</code>)

Пример Dockerfile для Node.js-сервиса

```
FROM node:18-alpine
```

```
WORKDIR /app
```

```
# Сначала копируем только манифест зависимостей —
# это позволяет Docker кэшировать слой с npm install
# и не переустанавливать пакеты при изменении только кода
```

```
COPY package*.json ./
```

```
RUN npm install
```

```
# Копируем остальной код приложения
```

```
COPY . .
```

```
EXPOSE 8000
```

```
CMD ["npm", "start"]
```



Tip

Почему `COPY package*.json` идёт перед `COPY . .` ?

Docker кэширует слои образа. Зависимости меняются редко, а код — часто. Если вынести установку зависимостей в отдельный слой до копирования кода, то при изменении только кода Docker возьмёт слой с `npm install` из кэша и не будет его перезапускать. Это существенно ускоряет сборку.

Docker Compose на примере микросервисного проекта

Когда приложение состоит из нескольких сервисов, управлять каждым контейнером вручную неудобно. `Docker Compose` позволяет описать всё приложение в одном файле `docker-compose.yml` и управлять им одной командой.

Структура проекта

Для ознакомления с тем, как работает Docker на практике, предлагается готовый пример микросервисного приложения на Node.js. Проект доступен по ссылке:

<https://github.com/darrrmr/docker-multi-service-example>.

```
project/
├── docker-compose.yml      # Описание всего стека сервисов
├── api_gateway/
│   ├── Dockerfile         # Инструкции сборки образа
│   ├── package.json       # Зависимости и скрипты запуска
│   └── index.js           # Код API Gateway с Circuit Breaker и агрегацией
├── service_users/
│   ├── Dockerfile
│   ├── package.json
│   └── index.js           # REST API для работы с пользователями
└── service_orders/
    ├── Dockerfile
    ├── package.json
    └── index.js           # REST API для работы с заказами
```

В проекте реализованы три сервиса:

- `api_gateway` — единственная точка входа, доступная снаружи (порт 8000). Принимает запросы, маршрутизирует их к нужным сервисам и реализует паттерн Circuit Breaker.
- `service_users` — сервис управления пользователями (CRUD). Не доступен напрямую снаружи — только через Gateway.
- `service_orders` — сервис управления заказами (CRUD). Также доступен только через Gateway.

Файл `docker-compose.yml`

```
services:
  api_gateway:
    build: api_gateway          # собираем образ из Dockerfile в папке
                                api_gateway
    ports:
      - "8000:8000"            # пробрасываем порт наружу: хост:контейнер
    environment:
      - NODE_ENV=production
    networks:
      - app-network

  service_users:
    build: service_users
    environment:
      - NODE_ENV=production
    networks:
      - app-network            # порт НЕ пробрасывается — сервис закрыт снаружи

  service_orders:
    build: service_orders
    environment:
      - NODE_ENV=production
    networks:
      - app-network

networks:
  app-network:
    driver: bridge              # все сервисы общаются между собой по имени
                                сервиса
```

Ключевые моменты:

- `build` — Docker соберёт образ из `Dockerfile` в указанной папке.

- `ports` — пробрасывает порт с хоста в контейнер. Только `api_gateway` имеет открытый порт — внутренние сервисы недоступны снаружи.
- `networks` — все сервисы в одной сети видят друг друга по имени: `http://service_users:8000`, `http://service_orders:8000`.
- `driver: bridge` — стандартная изолированная сеть Docker.

Логика работы сервисов

Сервис пользователей предоставляет REST API: получение всех пользователей, получение по ID, создание, обновление, удаление. Данные хранятся в памяти (переменная-объект), что означает их сброс при перезапуске контейнера.

Важная деталь: сервер слушает на адресе `0.0.0.0`, а не на `localhost`. Это обязательно для работы внутри контейнера — без этого другие контейнеры в сети не смогут до него достучаться.

```
// Слушаем на всех сетевых интерфейсах контейнера
app.listen(PORT, '0.0.0.0', () => {
  console.log(`Users service running on port ${PORT}`);
});
```

API Gateway принимает все входящие запросы и проксирует их к нужному сервису.

Адреса сервисов задаются через имена из `docker-compose.yml` — Docker сам разрешает их в IP-адреса:

```
const USERS_SERVICE_URL = 'http://service_users:8000';
const ORDERS_SERVICE_URL = 'http://service_orders:8000';
```

Circuit Breaker — паттерн защиты от каскадных сбоев. Если один из сервисов перестаёт отвечать, Circuit Breaker «открывается» и сразу возвращает клиенту резервный ответ (fallback), не ожидая таймаута. Через некоторое время делается тестовый запрос — если сервис восстановился, Circuit Breaker закрывается и запросы снова идут напрямую.

Три состояния Circuit Breaker:

- **Closed (закрыт)** — нормальная работа, запросы проходят.
- **Open (открыт)** — слишком много ошибок, запросы блокируются, возвращается fallback.
- **Half-Open (полукоткрыт)** — тестовый запрос для проверки восстановления сервиса.

API Aggregation — Gateway умеет объединять данные из нескольких сервисов в один ответ. Например, эндпоинт `/users/:id/details` параллельно запрашивает информацию

о пользователе и его заказах, после чего возвращает агрегированный результат. Оба запроса выполняются одновременно через `Promise.all`, что снижает суммарное время ожидания.

Запуск примера в WSL

Переход в директорию проекта

Откройте терминал WSL (найдите «Ubuntu» в меню «Пуск»). Диски Windows доступны через `/mnt/`. Если проект находится в `C:\Users\ВашеИмя\project`:

```
cd /mnt/c/Users/ВашеИмя/project
```

Буква диска пишется строчной буквой: `C:\` → `/mnt/c/`, `D:\` → `/mnt/d/`.

Убедитесь, что находитесь в нужной директории:

```
ls
# Должны видеть: docker-compose.yml  api_gateway  service_users
service_orders
```

Сборка и запуск контейнеров

```
docker compose up --build
```

- `up` — создаёт и запускает все контейнеры из `docker-compose.yml`.
- `--build` — принудительно пересобирает образы. Используйте при первом запуске или после изменения кода.

Когда всё запустится, в терминале появятся сообщения:

```
api_gateway-1      | API Gateway running on port 8000
service_users-1    | Users service running on port 8000
service_orders-1   | Orders service running on port 8000
```

Для запуска в фоновом режиме добавьте флаг `-d` (detached):

```
docker compose up -d --build
```

Просмотр запущенных контейнеров

docker ps

Пример вывода:

CONTAINER ID	IMAGE	COMMAND	STATUS	PORTS
a1b2c3d4e5f6	project-api_gateway	"npm start"	Up 2 minutes	0.0.0.0:8000->8000/tcp
b2c3d4e5f6a7	project-service_users	"npm start"	Up 2 minutes	
c3d4e5f6a7b8	project-service_orders	"npm start"	Up 2 minutes	

В графическом интерфейсе Docker Desktop запущенные контейнеры примую следующий формат:

Search

Only show running containers

		Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
▼		test	-	-	-	0.05%	29 seconds ago	
		api_gateway-1	97225f277d38	test-api_gateway	8000:8000	0.05%	29 seconds ago	
		service_orders-1	599af48b57a7	test-service_orders		0%	30 seconds ago	
		service_users-1	de1eef7d21ac	test-service_users		0%	30 seconds ago	

Тестирование API

После запуска выполните в терминале WSL:

```
# Проверка статуса Gateway
curl http://localhost:8000/status

# Создание пользователя
curl -X POST http://localhost:8000/users \
  -H "Content-Type: application/json" \
  -d '{"name": "Иван Иванов", "email": "ivan@example.com"}'

# Получение пользователя по ID
curl http://localhost:8000/users/1

# Создание заказа
curl -X POST http://localhost:8000/orders \
  -H "Content-Type: application/json" \
  -d '{"userId": 1, "product": "Ноутбук", "price": 75000}'

# Агрегированный запрос: пользователь + его заказы
curl http://localhost:8000/users/1/details
```

Проверка Circuit Breaker — остановите один из внутренних сервисов и убедитесь, что Gateway возвращает fallback:

```
docker compose stop service_users
curl http://localhost:8000/users/1
# Ожидаемый ответ: {"error": "Users service temporarily unavailable"}
```

Полезные команды Docker

Управление контейнерами

```
# Просмотр запущенных контейнеров
docker ps

# Просмотр всех контейнеров, включая остановленные
docker ps -a

# Остановить контейнеры (данные сохраняются)
docker compose stop

# Остановить и удалить контейнеры
docker compose down

# Остановить, удалить контейнеры и тома с данными
docker compose down -v

# Перезапустить один сервис
docker compose restart api_gateway
```

Warning

Осторожно: `docker compose down -v` удаляет все тома, включая данные баз данных. Используйте только при необходимости полной очистки.

Просмотр логов

```
# Логи конкретного сервиса
docker compose logs api_gateway

# Логи всех сервисов в режиме реального времени
docker compose logs -f
```

```
# Последние 50 строк логов
docker compose logs --tail=50 service_users
```

Работа с образами

```
# Просмотр всех образов на машине
docker images

# Скачать образ из Docker Hub (без запуска)
docker pull nginx:alpine

# Удалить образ
docker rmi nginx:alpine

# Удалить все неиспользуемые образы, контейнеры и сети
docker system prune
```

Вход внутрь контейнера

Для отладки можно «зайти» внутрь работающего контейнера и выполнять команды прямо в нем:

```
docker compose exec api_gateway sh
```

После этого вы окажетесь внутри контейнера. Для выхода введите `exit`.

Nginx как балансировщик в Docker Compose

В практике 22 Nginx настраивался как балансировщик нагрузки, когда backend-серверы запускались вручную на разных портах. В связке с Docker Compose эта схема становится значительно удобнее: каждый backend — отдельный контейнер, а Nginx обращается к ним по **именам сервисов**, без указания IP-адресов или портов хоста.

```
services:
  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf # монтируем конфиг с хоста
    networks:
      - app-network
```

```
backend1:
  build: ./backend
  networks:
    - app-network

backend2:
  build: ./backend
  networks:
    - app-network

networks:
  app-network:
    driver: bridge
```

Конфигурация `nginx.conf` при этом использует имена сервисов вместо `127.0.0.1`:

```
upstream backend {
  server backend1:3000;
  server backend2:3000;
}

server {
  listen 80;

  location / {
    proxy_pass http://backend;
  }
}
```

Docker сам разрешает `backend1` и `backend2` в IP-адреса соответствующих контейнеров. Это означает, что при масштабировании не нужно менять конфигурацию Nginx — достаточно добавить новый сервис в `docker-compose.yml`.

Несколько экземпляров одного образа

В Docker Compose можно запустить несколько экземпляров одного и того же образа через параметр `deploy.replicas` (в Swarm-режиме) или просто описав их как отдельные сервисы. Такой подход позволяет горизонтально масштабировать `backend` без изменения кода:

```
services:
  nginx:
    image: nginx:alpine
    ports:
```

```

    - "80:80"
volumes:
    - ./nginx.conf:/etc/nginx/nginx.conf
networks:
    - app-network
depends_on:
    - backend1
    - backend2
    - backend3

backend1:
    build: ./backend
    environment:
        - PORT=3000
        - SERVER_ID=1          # переменная для идентификации экземпляра в ответе
    networks:
        - app-network

backend2:
    build: ./backend
    environment:
        - PORT=3000
        - SERVER_ID=2
    networks:
        - app-network

backend3:
    build: ./backend
    environment:
        - PORT=3000
        - SERVER_ID=3
    networks:
        - app-network

networks:
    app-network:
        driver: bridge

```

```

upstream backend {
    server backend1:3000;
    server backend2:3000;
    server backend3:3000 backup;    # резервный сервер
}

server {

```

```
listen 80;

location / {
    proxy_pass http://backend;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
}
}
```

Расширенный docker-compose.yml с базой данных и Redis

Когда в приложении используются база данных и кэш, они также описываются в `docker-compose.yml` как отдельные сервисы:

```
services:
  api_gateway:
    build: api_gateway
    ports:
      - "8000:8000"
    networks:
      - app-network

  service_users:
    build: service_users
    environment:
      - DATABASE_URL=postgresql://user:password@db_users:5432/users_db
    depends_on:
      - db_users
      - cache
    networks:
      - app-network

  service_orders:
    build: service_orders
    environment:
      - DATABASE_URL=postgresql://user:password@db_orders:5432/orders_db
    depends_on:
      - db_orders
      - cache
    networks:
      - app-network

  db_users:
    image: postgres:17
    environment:
```

```

    POSTGRES_USER: user
    POSTGRES_PASSWORD: password
    POSTGRES_DB: users_db
  volumes:
    - postgres_data_users:/var/lib/postgresql/data # данные сохраняются
между перезапусками
  networks:
    - app-network

db_orders:
  image: postgres:17
  environment:
    POSTGRES_USER: user
    POSTGRES_PASSWORD: password
    POSTGRES_DB: orders_db
  volumes:
    - postgres_data_orders:/var/lib/postgresql/data
  networks:
    - app-network

cache:
  image: redis:7-alpine
  networks:
    - app-network

volumes:
  postgres_data_users:
  postgres_data_orders:

networks:
  app-network:
    driver: bridge

```

Важные элементы:

- `depends_on` — определяет порядок запуска: `service_users` стартует только после `db_users` и `cache`. Не гарантирует готовности базы данных, только старт контейнера.
- `volumes: postgres_data_users:` — именованный том. Данные PostgreSQL сохраняются на хосте и не теряются при перезапуске контейнера.
- `image: postgres:17` — используем готовый официальный образ без написания собственного `Dockerfile`.

Практическое задание

В рамках данного занятия необходимо реализовать веб-приложение с несколькими backend-серверами и балансировкой нагрузки через Nginx, развернув всё с помощью Docker Compose в среде WSL.

Задание является продолжением практики 22: вместо ручного запуска серверов на разных портах каждый компонент системы — отдельный Docker-контейнер.

В рамках выполнения задания требуется:

- установить WSL и Docker согласно инструкции из данного занятия;
- ознакомиться с примером проекта из репозитория — изучить структуру `Dockerfile` и `docker-compose.yml`, запустить пример и протестировать доступные эндпоинты;
- реализовать собственное приложение, включающее:
 - не менее двух идентичных backend-сервисов на Node.js (Express), каждый из которых отвечает на запрос `GET /` с указанием своего идентификатора (например, `{"server": "backend-1"}`);
 - Nginx в роли балансировщика нагрузки;
 - `docker-compose.yml`, описывающий все сервисы и объединяющий их в одну сеть;
 - `Dockerfile` для backend-сервиса;
 - конфигурационный файл `nginx.conf`;
- запустить весь стек командой `docker compose up --build` в WSL;
- проверить балансировку: при повторных запросах `curl http://localhost/` ответ должен поочерёдно приходить от разных backend-серверов;
- добавить настройки отказоустойчивости через `max_fails` и `fail_timeout` в конфигурации Nginx;
- остановить один из backend-контейнеров и убедиться, что Nginx перестаёт направлять на него запросы и продолжает обслуживать трафик через оставшиеся.

Формат отчета

В качестве ответа на задание необходимо прикрепить ссылку на репозиторий с реализованной практикой.